

nhnis-common [boot] (nhnis-common:feature/CSS+6:55)
src/main/java
nhnis.tw.commons
> apigw
> configuration
> context
> dto
> exception
> filter
> fos
> imagehog
> interceptor
> jwt
> message
> resolver
> transaction
> util
> JRE System Library [jvase-2.1]
> Project and External Dependencies
bin
> docs
> libs
> meta
> src
build.gradle
gradlew
gradlew.bat
README.md
settings.gradle
nm-service [boot] (development)

[파일] 응용 프로그램 [트] 신규 서비스 CRUD 자동 생성 (Spring Boot + MyBatis)

1. 역할 및 목표
본 프로그램은 Spring Boot + MyBatis 기반의 애플리케이션을 백엔드 아키텍처**입니다.
사용자가 제공하는 **데이터를 **데이터 (DDL)**와 **데이터를 바탕으로, **완전한 CRUD (Create, Read, Update, Delete) 서비스를 구현해야 합니다.

핵심 목표:
- 계층적 구조를 설계하여 **DTO, DAO, Service, Controller, XML**을 활용 생성합니다.
- **기본 프로젝트의 공통 컴포넌트** (패키지 구조, 데이터 규칙, 공통 패턴, 예외 처리)를
- **H2 (local)**와 **Oracle (dev/prod)** 모두에서 작동하는 SQL을 작성합니다.

2. 입력 데이터 (사용자가 제공)

2.1 프로젝트 컨텍스트
- 프로젝트명: **{프로젝트명, 예: pdmk}
- 도메인 패키지: **{패키지명, 예: nhns.mk.co.a}
- 기본 코드: **{기호코드, 예: mkpca9999}
- 데이터소스: **{RDB 또는 ADW}

2.2 데이터 형식 (DDL)
* (아래의 데이터를 구조를 입력하세요)
\sql
-- 테이블
CREATE TABLE {테이블명} (
SÑO NUMBER(14) PRIMARY KEY, -- 원본번호 (시퀀스)
{컬럼명} VARCHAR2(20), -- 일련번호
{컬럼명} VARCHAR2(60), -- 설명
{컬럼명} NUMBER(7), -- 수량
RG_BRC CHAR(6), -- 등록사구코드
RG_EMP_ENO VARCHAR2(9), -- 등록직원번호
RG_DT CHAR(8), -- 등록일자
FL_PRC_YN CHAR(1) DEFAULT 'Y', -- 처리여부
LSCHG_ENO VARCHAR2(9), -- 최종변경자
LSCHG_DT CHAR(8), -- 최종변경일
ETL_WK_DTM DATE -- ETL 일시
);

2.3 기본 연세 (CRUD 요구사항)
1. **조회 (S0): **{검색조건}으로 목록 조회, {출력필드}, 포함, 저장.
2. **등록 (S1): **{입력필드}, {신규 등록}, {데이터베이스}, 동시 저장.
3. **상세조회 (S4): **{SÑO}, {상세 정보 + 자식 목록 조회}, 포함, 저장.
4. **수정 (S2): **{SÑO}, {기반 수정}, {데이터베이스}, 전체 갱신 (Delete + Insert).
5. **삭제 (S3): **{SÑO List}, {기반 삭제/물리 삭제}.

3. 구현 가이드라인 (반드시 준수)

```

## 3.1 패키지 및 네임 공간
**패키지: ** {도메인 패키지}.controller, .service, .dao, .dto
**를 배울: ** {컨트롤러} {서비스} {컨트롤러/서비스/DAO}
**DTO: **
인원: {컨트롤러} {서비스} {DTO인}
출력: {컨트롤러} {서비스} {DTOout}
서드: {컨트롤러} {서비스} {DTOsub0}
**메서드: **
Service: {컨트롤러} {서비스} {예: 'mkpca9999S1'}
DAO: {컨트롤러} {서비스} {예: 'mkpca9999S1 S0'}
Controller: {컨트롤러} {서비스} {예: 'mkpca9999S1'}

```

```
4. **Controller 클래스:**
- {코드}controller.java (엔드포인트: /s0, /s1, /s2, /s3, /s4)
5. **XML 매핑 파일:**
- {데이터소스}/{패키지}/{코드}-ORA.xml (모든 SQL 포함)
```

```
6. **H2 스키마/데이터:**
- `schema.sql`: 테이블 생성 (Oracle 시퀀스 포함용 DDL 포함).
- `data.sql`: 테스트용 데이터 (5건 이상).
```

5. 출력 형식 요구사항

```
- **파일명 규칙:** 각 코드 블록 상단에 파일명을 표시하십시오.
- **공백 관련 코드:** import 문, 클래스 정의, 메서드 구현을 모두 포함하십시오.
- **주석:** 주공, 파라마미터, 회염, SQL 주석을 포함하십시오.
```

```
1. DTO (S0, S1, S2, S3, S4)
```

```
2. DAO
3. Service
4. Controller
5. XML
6. schema.sql / data.sql 엔드포인트
```

6. 시작 지시

"위 가이드라인을 바탕으로, 아래 입력 데이터를 사용하여 **전체 CRUD 서비스**를 생성하십시오.

```
- **프로젝트:** {프로젝트명}
- **기억코드:** {기억코드}
- **테이블:** {테이블명}
- **세션:** {세션명}
```

생성 지시

사용 예시 (사용자가 이 프로젝트를 생성할 때는 필수)

사용자 입력

```
> "위 프로젝트를 사용하여 'pdmk' 프로젝트에 'mkpca9999' (마케팅 캠페인 관리) 기능을 만들어줘."
> - **테이블:** 'TB CR CM MKT CAMP' (SQNO, CAMP_NM, START_DT, END_DT, OBJ_CNT, RG_BRC...)
> - **자식:** 'TB CR CM MKT CAMP TGT' (SQNO, CUSNO, TGT_YN...)
> - **기억:**
> - S0: 사무소별 캠페인 목록 조회
> - S1: 신규 캠페인 등록 (타겟 고객 목록 포함)
> - S2: 캠페인 수정 (타겟 고객 전체 갱신)
> - S3: 캠페인 삭제 (논리 삭제: FL_PRC_YN='N')
> - S4: 캠페인 상세 조회"
```

이 프로젝트를 사용하면 AI는 **관련 패턴**으로 **완전 CRUD 코드를** 생성할 수 있습니다.

```

package nhnis.fw.commons.apigw;

import java.time.Duration;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.http.client.reactive.ReactorClientHttpConnector;
import org.springframework.web.reactive.function.BodyInserters;
import org.springframework.web.reactive.function.client.WebClient;

import io.netty.channel.ChannelOption;
import nhnis.fw.commons.apigw.dto.ApiGatewayDto;
import reactor.core.publisher.Mono;
import reactor.netty.http.client.HttpClient;

public class ApiGatewayHandler {

    private ClientHttpConnector clientHttpConnector;

    @Value("${apigw.connect-timeout:30}")
    private static int CONNECT_TIMEOUT;

    @Value("${apigw.response-timeout:30}")
    private static int RESPONSE_TIMEOUT;

    private static final String SEND_TYPE = "W";

    @Bean
    private ClientHttpConnector connector() {
        if (clientHttpConnector == null) {
            clientHttpConnector = (ClientHttpConnector)new
            ReactorClientHttpConnector(httpClient());
            return clientHttpConnector;
        } else {
            return clientHttpConnector;
        }
    }

    private HttpClient httpClient() {
        return HttpClient.create().option(ChannelOption.CONNECT_TIMEOUT_MILLIS,
        CONNECT_TIMEOUT * 1000).responseTimeout(Duration.ofMillis(RESPONSE_TIMEOUT * 1000));
    }

    public Object call(ApiGatewayDto dto) {
        Mono<Object> response =
        WebClient.builder().clientConnector(clientHttpConnector).baseUrl(dto.getUrl()).build().post()
        .header(ApiGatewayHeaderType.CONTENT_TYPE, CONTENT_TYPE).get(),
        dto.getContentId(),
        dto.getInterfaceId(),
        header(ApiGatewayHeaderType.INTERFACE_ID, get(),
        header(ApiGatewayHeaderType.RECV_TEXT_NAME, get(),
        dto.getRecvTrxName())
        .header(ApiGatewayHeaderType.RECV_TYPE, get(), dto.getRecvType())
        .header(ApiGatewayHeaderType.REPLY_TYPE, get(), dto.getReplyType())
        .header(ApiGatewayHeaderType.SEND_TYPE, SEND_TYPE)
        .body(BodyInserters.fromValue(dto.getBody()).retrieve())
        .bodyToMono(Object.class);
        Object result = response.block();
        return result;
    }
}

```

}

}

```

package nhnis.fw.commons.apigw;

public enum ApiGatewayHeaderType {

    INTERFACE_ID("interface-id"),
    CONTENT_TYPE("content-type"),
    RECV_TEX_NAME("recv-trx-name"),
    RECV_TYPE("recv-type"),
    SEND_TYPE("send-type"),
    REPLY_TYPE("reply-type");

    private final String value;

    ApiGatewayHeaderType(String value) {
        this.value = value;
    }

    public String get() {
        return value;
    }
}

```

```

package nhnis.fw.commons.apigw.dto;

import com.tmax.superspring.dto.DataObject;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Collections;
import java.util.Map;
import com.tmax.superspring.dto.engine.dto.record.common.FieldProperty;
import java.util.stream.Collectors;

@jakarta.annotation.Generated(
    value = "com.tmaxsoft.sts4.codegen.dto.DtoGenerator",
    date = "26. 7. 21. 오후 11:15",
    comments = "ApiGatewayDto"
)
public class ApiGatewayDto extends DataObject
{
    private static final long serialVersionUID = 1L;

    private String url = null;

    public String getUrl() {
        return url;
    }

    public void setUrl(String url) {
        if(url == null) {
            this.url = null;
        } else {
            this.url = url;
        }
    }

    private String contentType = null;

    public String getContentType() {
        return contentType;
    }

    public void setContentType(String contentType) {
        if(contentType == null) {
            this.contentType = null;
        } else {
            this.contentType = contentType;
        }
    }

    private String interfaced = null;

    public String getInterfaced() {
        return interfaced;
    }

    public void setInterfaced(String interfaced) {
        if(interfaced == null) {
            this.interfaced = null;
        }
    }
}

```



```

        } else {
            this.interfaceId = interfaced;
        }
        private String recvTxName = null;
        public String getRecvTxName() {
            return recvTxName;
        }
        public void setRecvTxName(String recvTxName) {
            if(recvTxName == null) {
                this.recvTxName = null;
            } else {
                this.recvTxName = recvTxName;
            }
        }
        private String recvType = null;
        public String getRecvType() {
            return recvType;
        }
        public void setRecvType(String recvType) {
            if(recvType == null) {
                this.recvType = null;
            } else {
                this.recvType = recvType;
            }
        }
        private String replyType = null;
        public String getReplyType() {
            return replyType;
        }
        public void setReplyType(String replyType) {
            if(replyType == null) {
                this.replyType = null;
            } else {
                this.replyType = replyType;
            }
        }
        private String body = null;
        public String getBody() {
            return body;
        }
        public void setBody(String body) {
            if(body == null) {
                this.body = null;
            } else {
                this.body = body;
            }
        }
    }

```

```

    }

    public Object clone() {
        ApiGatewayDto copyObj = new ApiGatewayDto();
        copyObj.clone(this);
        return copyObj;
    }

    public void clone(DataObject _apiGatewayDto) {
        if(this == _apiGatewayDto)
            return;
        ApiGatewayDto _apiGatewayDto = (ApiGatewayDto) _apiGatewayDto;
        this.setUrl(_apiGatewayDto.getUrl());
        this.setContentType(_apiGatewayDto.getContentType());
        this.setInterfaceId(_apiGatewayDto.getInterfaceId());
        this.setRecvTxName(_apiGatewayDto.getRecvTxName());
        this.setRecvType(_apiGatewayDto.getRecvType());
        this.setReplyType(_apiGatewayDto.getReplyType());
        this.setBody(_apiGatewayDto.getBody());
    }

    public String toString() {
        StringBuilder buffer = new StringBuilder();
        buffer.append("url : ").append(_url).append("\n");
        buffer.append("contentType : ").append(contentType).append("\n");
        buffer.append("interfaceId : ").append(interfaceId).append("\n");
        buffer.append("recvTxName : ").append(recvTxName).append("\n");
        buffer.append("recvType : ").append(recvType).append("\n");
        buffer.append("replyType : ").append(replyType).append("\n");
        buffer.append("body : ").append(body).append("\n");
        return buffer.toString();
    }

    private static final Map<String, FieldProperty> fieldPropertyMap;

    static {
        fieldPropertyMap = new java.util.LinkedHashMap<String, FieldProperty> (7);
        fieldPropertyMap.put("url", FieldProperty.builder()
            .setPhysicalName("url")
            .setLogicalName("url")
            .setType(FieldProperty.Type.OBJECT_STRING)
            .setDecimal(-1)
            .setNullable(true)
            .setEncrypt(false)
            .build());
        fieldPropertyMap.put("contentType", FieldProperty.builder()
            .setPhysicalName("contentType")
            .setLogicalName("contentType")
            .setType(FieldProperty.Type.OBJECT_STRING)
            .setDecimal(-1)
            .setNullable(true)
            .setEncrypt(false)
            .build());
        fieldPropertyMap.put("interfaceId", FieldProperty.builder()
            .setPhysicalName("interfaceId")
            .setLogicalName("interfaceId")
            .setType(FieldProperty.Type.OBJECT_STRING)
            .setDecimal(-1)
            .setNullable(true)
            .setEncrypt(false)
            .build());
    }

```

```
import com.tmax.superstring.dto.engine.exception.UnmarshalException;
import com.tmax.superstring.dto.engine.exception.MarshallException;
import com.tmax.superstring.dto.DataObject;
import java.util.ArrayList;
import java.util.List;
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import com.google.gson.stream.JsonReader;
import com.google.gson.stream.JsonWriter;
import com.google.gson.JsonToken;
import org.w3c.dom.Node;
```

```
package nhnis.fw.commons.apigw.dto;

}

public Map<String, FieldProperty> getFieldPropertyMap() {
    return Collections.unmodifiableMap(fieldPropertyMap);
}
```

```
fieldPropertyMap.put("body", FieldProperty.builder()
    .build());
fieldPropertyMap.put("body", FieldProperty.builder()
    .setPhysicalName("body")
    .setLogicalName("body")
    .setType(FieldProperty.Type.OBJECT_STRING)
    .setDecimal(-1)
    .setNullable(true)
    .setEncrypt(false)
    .build());
fieldPropertyMap.put("replyType", FieldProperty.builder()
    .setPhysicalName("replyType")
    .setLogicalName("replyType")
    .setType(FieldProperty.Type.OBJECT_STRING)
    .setDecimal(-1)
    .setNullable(true)
    .setEncrypt(false)
    .build());
fieldPropertyMap.put("recvType", FieldProperty.builder()
    .setPhysicalName("recvType")
    .setLogicalName("recvType")
    .setType(FieldProperty.Type.OBJECT_STRING)
    .setDecimal(-1)
    .setNullable(true)
    .setEncrypt(false)
    .build());
fieldPropertyMap.put("recvTxName", FieldProperty.builder()
    .setPhysicalName("recvTxName")
    .setLogicalName("recvTxName")
    .setType(FieldProperty.Type.OBJECT_STRING)
    .setDecimal(-1)
    .setNullable(true)
    .setEncrypt(false)
    .build());
```